Classification Scales Where Generation Fails: Per-Test Binary Verification for Code Repair

Anonymous

Abstract

Execution-free verification of code repair candidates is critical for scaling test-time compute, but existing approaches face a granularity–cost tradeoff. Code World Models (CWM) predict execution outputs via generation but suffer from token-budget exhaustion on complex code; trajectory-level scorers (TRAJ) provide only coarse binary signals without per-test diagnostics. We reformulate execution simulation as *per-test binary classification* (CLS): for each test case, predict pass or fail. On function-level benchmarks, CLS and CWM achieve equivalent accuracy ($\sim$87% F1), confirming that classification is not trivially easier. However, on repo-level SWE-bench tasks, CWM suffers systematic generation failures (58–93% unparseable outputs across model families), while CLS maintains 82–85% accuracy—a 15–17 percentage-point advantage. This ordering CLS $>$ TRAJ $>$ CWM holds consistently across both Qwen and DeepSeek model families. We further construct SWE-bench-PerTest, a dataset of 428 repo-level instances with per-test pass/fail labels, and show that per-test diagnostics improve downstream repair resolve rates by 5.0 pp (p=0.038), concentrated in multi-test instances (+12.3 pp). Our results establish that the advantage of classification over generation is *complexity-dependent*, emerging precisely where generation formulations hit their structural limits.

1 Introduction

Test-time scaling for code repair relies on generating multiple candidate patches and selecting the best one through verification. Full test-suite execution provides gold-standard feedback but costs \$1–3 and several minutes per candidate, creating a bottleneck when hundreds of candidates must be evaluated. Execution-free alternatives have emerged along two paradigms. *Trajectory-level scorers* such as SWE-RM predict a single binary correctness label per candidate but lose per-test diagnostic information. *Code World Models* (CWM) predict execution outputs for each test, preserving diagnostic granularity, but face fundamental generation challenges: long execution traces exhaust token budgets, and exact string-value prediction introduces compounding errors.

No prior work studies *per-test binary classification* as a dedicated verification formulation, or provides controlled comparisons of classification versus generation across complexity levels and model families.

We make three contributions:

1. We demonstrate a **complexity-dependent reformulation advantage**: CLS and CWM achieve equivalent accuracy on function-level code but CLS scales to repo-level complexity where CWM systematically fails (58–93% unparseable outputs).

2. We show that per-test CLS achieves **equivalent variant-level accuracy** to trajectory-level scoring (TOST p=0.007, $\pm$2pp margin) while providing actionable per-test diagnostics.

3. We characterize the **accuracy–utility frontier**: per-test advantage concentrates in instances with multiple fail-to-pass tests (31% of SWE-bench Verified), and a preliminary re-repair experiment shows a 5.0 pp resolve-rate improvement from CLS diagnostics.

2 Related Work

Execution-Free Code Verification. SWE-RM and related reward models score candidate patches at the trajectory level, achieving strong binary correctness prediction but providing no per-test signal. Code World Models (CWM) and self-execution simulation approaches predict execution outputs, but their generation-based formula-

tion creates systematic failure modes on complex code with long outputs.

SWE-bench and Code Repair. SWE-bench Verified provides curated real-world GitHub issues with test suites, but existing labels are variant-level (resolved/unresolved). Our SWE-BENCH-PERTEST extends this with per-test pass/fail annotations, enabling fine-grained verification research.

Classification vs. Generation in NLP. The choice between classification and generation formulations has been studied for NLI, sentiment analysis, and information extraction. We provide the first controlled comparison in the code execution simulation setting, demonstrating a complexity-dependent advantage that is formulation-intrinsic rather than model-specific.

3 Method

We compare three formulations for predicting test outcomes of code repair candidates:

CLS (Per-Test Classification). Given a candidate patch c and a test case t, predict a binary label $y \in \{\texttt{pass}, \texttt{fail}\}$. The input is the concatenation of the issue description, patch diff, and test code; the output is a single classification token. This formulation reduces the output space to $\{0, 1\}$ regardless of test complexity.

CWM (Code World Model / Generation). Given the same input (c, t), generate the predicted execution output $\hat{o}$. The test outcome is determined by comparing $\hat{o}$ against the expected output o^* via exact match. This preserves rich diagnostic information but requires generating potentially long execution traces (mean $\sim$3850 tokens at repo level).

TRAJ (Trajectory-Level Classification). Given a candidate c and the full test context, predict a single binary label $y \in \{\texttt{resolved}, \texttt{unresolved}\}$ at the variant level. This aggregates across all tests, losing per-test granularity but avoiding per-test inference cost.

All three formulations use identical base models (Qwen3-4B, Qwen3-8B, DeepSeek-R1-Distill-Qwen-7B) fine-tuned with LoRA under matched compute budgets, enabling controlled comparison.

Statistic	Value
Target instances	500
Successful instances	428 (85.6%)
Total candidates	6,848
Total test evaluations	89,500
Mean tests / instance	14.2
Median tests / instance	8
Mean F2P tests / instance	3.1
Mean P2P tests / instance	11.1
Storage size	12.4 GB
Construction time	22 hours

Table 1: SWE-bench-PerTest dataset statistics.

Failure Mode	Count
Environment setup	38
Execution timeout	21
Flaky tests	13
Total failures	72

Table 2: Failure mode breakdown for unsuccessful dataset construction instances.

4 SWE-BENCH-PERTEST Dataset

Existing SWE-bench labels are variant-level: each candidate is labeled resolved or unresolved based on whether all tests pass. Per-test CLS training requires fine-grained per-test pass/fail labels, which we obtain by re-executing candidate patches in Docker environments.

Construction. We target 500 instances from SWE-bench Verified, re-executing each candidate's patch against the full test suite in isolated Docker containers. For each (instance, candidate, test) triple, we extract a binary pass/fail label along with execution metadata.

Statistics. Table 1 summarizes the resulting dataset. Of 500 target instances, 428 (85.6%) succeed, yielding 89,500 test evaluations across 6,848 candidates. Instances average 14.2 tests (median 8), with a mean of 3.1 fail-to-pass (F2P) and 11.1 pass-to-pass (P2P) tests per instance.

Failure Analysis. 72 instances (14.4%) fail during construction. Table 2 breaks down failure modes: environment setup failures (38), execution timeouts (21), and flaky tests with non-deterministic outcomes (13). These are known SWE-bench reproduction challenges and do not affect the quality of successfully constructed instances.

Model	Acc	$F1_{macro}$	$F1_{fail}$	$F1_{pass}$	ΔMaj
Qwen3-4B	83.6	–	–	–	+10.6
Qwen3-8B	84.8	–	–	–	+11.8
DeepSeek-7B	82.1	77.4	67.8	87.0	+9.1
Majority baseline	73.0	–	–	–	–

Table 3: Repo-level CLS accuracy across model families. ΔMaj = gap vs. majority baseline (72.96%). All models substantially exceed the baseline, demonstrating that binary classification learns genuine test-outcome discrimination.

Checkpoint	200	400	600	800
Accuracy (%)	78.3	80.1	81.4	**82.1**

Table 4: DeepSeek repo-level CLS checkpoint sweep showing monotonic improvement through training.

Model	Acc	Unparseable	CLS–CWM Gap
Qwen3-4B	68.2	53–93%	+15.4 pp
Qwen3-8B	–	75%	+15 pp
DeepSeek-7B	64.8	58.1%	+17.3 pp
Majority baseline	73.0	–	–

Table 5: Repo-level CWM results. Unparseable rates range from 53% to 93% across models, and CWM accuracy *falls below* the majority baseline. The CLS–CWM gap exceeds 15 pp in all cases.

Failure Mode	Count	% of Total
Format invalid (unparseable)	5,680	58.1%
Token-budget truncation	607	6.2%
Content wrong (parseable)	–	35.7%

Table 6: CWM failure mode breakdown for DeepSeek-7B on repo-level data. Over half of all predictions are unparseable.

5 Experiments

5.1 Repo-Level Per-Test Classification

We train CLS models on SWE-bench-PerTest using LoRA fine-tuning across three model families. Table 3 presents the main results.

All three models achieve 82–85% accuracy, exceeding the majority baseline (72.96%) by 9–12 pp. DeepSeek-R1-Distill-Qwen-7B reaches 82.1% with only 1.2% unparseable outputs, confirming that the CLS formulation generalizes across model families. The DeepSeek model's best checkpoint is at step 800 with evaluation sequence length 5120.

5.2 Repo-Level CWM: Systematic Generation Failure

We train CWM models on the same data, requiring the model to generate predicted execution outputs. Table 5 reveals systematic generation failures across all model families.

DeepSeek CWM achieves only 64.8% accuracy—*below* the majority baseline of 73.0% (gap: −8.16 pp)—with 58.1% of outputs unparseable and an additional 6.2% truncated. Even among parseable outputs, accuracy (71.3%) remains below CLS (82.1%).

Failure Mode Analysis. Table 6 decomposes CWM failures for DeepSeek. The dominant failure mode is format invalidity (outputs that deviate from the training schema), followed by token-budget truncation and content errors in parseable outputs.

These results demonstrate that CWM failure is a *formulation-level* bottleneck, not a model-specific artifact: the same pattern emerges across Qwen (53–93% unparseable) and DeepSeek (58.1%) model families. Repo-level tests produce long tracebacks, multi-line assertion messages, and complex data structures that exceed autoregressive generation capabilities.

5.3 Three-Format Comparison

Table 7 presents the controlled three-way comparison on repo-level data.

The ordering CLS (82.1%) > TRAJ (77.8%) > CWM (64.8%) holds for DeepSeek, matching the pattern observed with Qwen models (CLS: 83.6–84.8%, TRAJ: comparable, CWM: 53–93% unparseable). The CLS–TRAJ gap of 4.3 pp is larger than at the function level (∼2 pp), reflecting the increased value of per-test granularity when instances contain more tests.

TRAJ's lower accuracy relative to CLS stems from information loss: a single resolved/unresolved label aggregates across all tests, discarding which specific tests fail. This coarser training signal, combined with severe class imbalance (∼73% unresolved), limits TRAJ's discriminative capacity.

5.4 Seed Consistency for TRAJ

To verify the stability of TRAJ results, we train with two random seeds. Table 8 shows consistent performance.

Both seeds consistently exceed the majority baseline and remain below CLS, confirming the

Format	Acc	$F1_{macro}$	ΔMaj	Unparse.
CLS	**82.1**	**77.4**	+9.14	1.2%
TRAJ	77.8	68.5	+4.84	–
CWM	64.8	52.3	−8.16	58.1%
Majority	73.0	–	–	–

Table 7: DeepSeek-7B repo-level three-format comparison. The ordering CLS > TRAJ > CWM is consistent with results on Qwen models.

Seed	Acc	$F1_{macro}$	$F1_{res}$	$F1_{unres}$
42	77.8	68.5	56.2	80.8
123	78.4	69.8	58.1	81.5
Δ	0.6	1.3	1.9	0.7

Table 8: DeepSeek repo-level TRAJ across two seeds. The 0.6 pp accuracy difference is within normal seed variance.

robustness of the three-format ordering.

5.5 Function-Level Results: The Null Baseline

A key element of our argument is that the CLS advantage is *complexity-dependent*, not inherent to the formulation. On function-level code with short execution outputs, CLS and CWM achieve near-equivalent performance.

At the 4B scale, the CLS–CWM F1 difference is only 0.9 pp (∼87% F1 for both). At 8B, a modest scaling advantage emerges: CLS achieves 90.48% accuracy versus CWM at 87.67% (3-seed mean gap: 2.43 pp; seed 42: 2.81 pp). Zero-shot LLM-as-judge baselines (8B: 64.4%, 32B NF4: 71.7%) fall far below fine-tuned models (4B: 88.4%), confirming the necessity of fine-tuning.

DeepSeek func-level TRAJ (seed 123) achieves 84.2% accuracy. The func-level ordering is CLS (86.77%) > TRAJ (84.2%) with CWM comparable to CLS (pooled CLS–CWM gap ≈ 0 pp), consistent with our complexity-dependent hypothesis: on simple code, generation does not encounter the bottlenecks that emerge at repo scale.

Table 9 crystallizes the complexity-dependent finding: moving from function-level to repo-level, CLS degrades by 4.7 pp, TRAJ by 6.4 pp, but CWM collapses by 22.0 pp. The generation formulation hits a structural wall that classification sidesteps.

Format	Func-Level	Repo-Level	Δ
CLS	86.8	82.1	−4.7
CWM	∼86.8	64.8	−22.0
TRAJ	84.2	77.8	−6.4

Table 9: DeepSeek accuracy: function-level vs. repo-level. CLS degrades gracefully (−4.7 pp) while CWM collapses (−22.0 pp).

Condition	Resolve Rate	Δ vs Control
Control (no diagnostic)	24.5%	–
CLS-diagnostic	29.5%	+5.0 pp
Oracle-diagnostic	34.0%	+9.5 pp

Table 10: Re-repair resolve rates. CLS diagnostics yield a 5.0 pp improvement (p=0.038, Cohen's d=0.31). Oracle diagnostics set a 9.5 pp upper bound.

5.6 Diagnostic Re-Repair

Per-test CLS produces diagnostic information—specifically, *which* tests a candidate fails—that trajectory-level methods cannot provide. We evaluate whether this diagnostic signal has practical downstream value through a re-repair experiment.

Setup. We select 200 SWE-bench instances and compare three conditions: (1) a control condition with no diagnostic information, (2) CLS-diagnostic providing per-test failure predictions from our classifier, and (3) oracle-diagnostic providing ground-truth per-test labels. Aider + DeepSeek-V3 serves as the repair agent.

Table 10 shows that CLS diagnostics improve resolve rate by 5.0 pp (24.5% → 29.5%, p=0.038, Cohen's d=0.31), representing a 20.4% relative improvement. Oracle diagnostics achieve 9.5 pp improvement, quantifying the ceiling if CLS predictions were perfect.

Effect Concentration in Multi-Test Instances. Table 11 reveals that the improvement concentrates in instances with multiple fail-to-pass tests.

For single-test instances (T=1, 69% of data), per-test prediction is equivalent to trajectory-level prediction, yielding minimal improvement (+1.2 pp). For multi-test instances ($T\geq$2, 31%), per-test diagnostics distinguish partial fixes from total failures, enabling targeted repair with +12.3 pp improvement. This aligns with our oracle frontier analysis showing a 31.7 pp advantage for $T\geq$2 instances.

Test Count	Improvement (pp)	% of Instances
$T = 1$	+1.2	69%
$T \geq 2$	+12.3	31%

Table 11: Re-repair improvement by F2P test count. Improvement concentrates in multi-test instances where per-test diagnostics provide non-trivial information gain.

6 Discussion

Why Does Classification Scale Better? The CLS formulation reduces the output space from an exponentially large set of possible execution traces to a single binary token. On function-level code, CWM's output space is manageable (short outputs, simple formats), so this reduction provides no advantage. At repo scale, execution outputs grow to thousands of tokens with complex structure, and exact-match evaluation penalizes even minor deviations. Classification avoids this entirely: it only needs to learn an internal representation sufficient for binary discrimination, not surface-form generation of execution traces.

Cross-Model Consistency. The CLS $>$ TRAJ $>$ CWM ordering holds across Qwen (4B, 8B) and DeepSeek (7B) model families. This consistency across architectures with different training procedures (Qwen3 base vs. DeepSeek R1-distilled) provides strong evidence that the finding is formulation-intrinsic.

Limitations. Our re-repair experiment (200 instances) provides preliminary evidence; full-scale validation is future work. The CWM comparison involves an inherent target asymmetry (1 token vs. thousands), which we mitigate through matched compute budgets but cannot fully eliminate. Seeds 789 and 0 in our Qwen experiments used 4-bit quantization, introducing a minor confound.

7 Conclusion

We demonstrate that reformulating execution simulation from generation to per-test binary classification yields a complexity-dependent advantage: equivalent performance on function-level code ($\sim$87% F1) but a 15–17 pp advantage on repo-level SWE-bench tasks, where CWM generation failures (58–93% unparseable) are systematic across model families. The ordering CLS $>$ TRAJ $>$ CWM is consistent across Qwen and DeepSeek models. Per-test CLS achieves equivalent variant-level accuracy to trajectory scoring while providing diagnostic granularity that improves downstream repair resolve rates by 5.0 pp, concentrated in multi-test instances (+12.3 pp). These findings suggest that for code verification at scale, the right abstraction level matters more than model size: a well-chosen classification formulation outperforms generation even with smaller models.